School of Computer Science and Engineering

December 2025

AUTOMATED CROSS CLOUD DISASTER RECOVERY FOR OBJECT STORAGE USING SERVERLESS SCRIPTING

Submitted in partial fulfillment for the award of the degree of

B.Tech (Hons)

by

N SUKHITHA BHUSHAN (1RVU23CSE293)

CHAKRIKA YEDLURI (1RVU23CSE127)

R KEERTHANA PRASAD (1RVU23CSE363)

SANSKRITI TONDIHAL (1RVU23CSE409)

Guided by

Prof. Aparna R

School of Computer Science and Engineering

RV University

RV Vidyaniketan,8th Mile, Mysore Road, Bengaluru, Karnataka, India - 560059

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

CERTIFICATE

This is to certify that the project work titled **"**AUTOMATED CROSS CLOUD DISASTER RECOVERY FOR OBJECT STORAGE USING SERVERLESS SCRIPTING**"** is performed by N Sukhitha Bhushan **(1RVU23CSE293)** Chakrika Yedluri **(1RVU23CSE127)** R Keerthana Prasad **(1RVU23CSE363)** Sanskriti Tondihal **(1RVU23CSE409),** a bonafide students of B.Tech (H) at the School of Computer Science and Engineering, RV university, Bengaluru in partial fulfillment for the award of degree B.Tech (H) in Computer Science & Engineering , during the Academic year **2025-2026**.

Prof.Aparna R	**Dr. Sudhakar K N**	**Dr. G Shobha**
Guide	**[B.Tech(H)]**	**[B.Tech(H)]**
Professor	Program Director	Dean
SoCSE	SoCSE	SoCSE
RV University	RV University	RV University
Date:01/12/2025	Date:01/12/2025	Date:01/12/2025

Signature of the Mentor

Signature of the Guide

Name of the Examiner

Signature of Examiner

1.

1.

2.

2.

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

DECLARATION

I hereby declare that the report entitled "AUTOMATED CROSS CLOUD DISASTER RECOVERY FOR OBJECT STORAGE USING SERVERLESS SCRIPTING" submitted by N Sukhitha Bhushan, Chakrika Yedulri, R Keerthana Prasad, Sanskriti Tondihal, for the award of the degree B.Tech (H) in RV University is a record of Bonafide work carried out by me under the supervision of Prof. Aparna R.

I further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Bangalore

Date : 01/12/2025

Signature of the Candidates

N Sukhitha Bhushan

Chakrika Yedluri

R Keerthana Prasad

Sanskriti Tondihal

ABSTRACT

The multi-cloud landscape necessitates robust and cost-effective Disaster Recovery (DR) solutions for object storage. Traditional cross-cloud backup methods are often complex, costly, and, relying on continuous replication, introduce significant inefficiencies: redundant data transfers, excessive bandwidth consumption, and increased storage overhead.

This project first establishes a foundational serverless, event-driven cross-cloud DR solution between Amazon S3 (AWS) and Google Cloud Storage (GCP). The architecture uses AWS Lambda, triggered by S3 PUT events, to securely replicate objects, leveraging AWS Secrets Manager for GCP credentials and a custom Lambda Layer for cross-cloud communication.

Crucially, the solution is optimized with a Content-Aware Differential Replication Strategy (CADRS). This strategy enhances efficiency by having the Lambda function compute an SHA-256 hash for each object and track changes in DynamoDB. Replication to GCP is executed only when content is new or modified, eliminating unnecessary transfers. The system integrates a Streamlit dashboard for unified operational monitoring.

Through experimental validation, this unified serverless framework demonstrates a practical, scalable, and highly cost-aware mechanism for protecting object storage across heterogeneous cloud providers. By integrating AWS and GCP services through serverless scripting, this project establishes a robust cross-cloud disaster recovery model that effectively reduces redundant storage operations and minimizes cross-cloud network usage while maintaining high reliability. This work contributes a superior, efficient, serverless approach to multi-cloud DR engineering.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Aparna R, Professor, School of Computer Science and Engineering, RV University, for his/her constant guidance, continual encouragement, understanding; more than all, he taught me patience in my endeavor. My association with him / her is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and expert in the field of Cloud Computing.

I would like to express my gratitude to Prof Dr Dwarika Prasad Uniyal, in charge Vice Chancellor RV University, and Prof Dr Shobha G, Dean SoCSE, RV University, for providing an environment to work in and for his inspiration during the tenure of the course.

In a jubilant mood I express ingeniously my whole-hearted thanks to **Dr. Sudhakar K N** Program Director, **[B.Tech(H)]** SoCSE, RV University, all teaching staff and members working as limbs of our university for their not-self-centered enthusiasm coupled with timely encouragements showered on me with zeal, which prompted the acquirement of the requisite knowledge to finalize my course study successfully. I would like to thank my parents for their support.

I would like to extend my sincere thanks to my mentor, Aparna R, Professor, School of Computer Science and Engineering, RV University, for their unwavering support, thoughtful guidance, and motivational presence throughout the course of this project. Their mentorship not only enriched my learning but also inspired me to approach challenges with confidence and clarity. Working with [him/her/them] has been a meaningful experience, and I am truly grateful for the valuable time and insights shared with me during this journey.

It is indeed a pleasure to thank my friends who persuaded and encouraged me to take up and complete this task. At last but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly toward the successful completion of this project.

Place: Bangalore

Date:20/07/2025

Name of the students

N Sukhitha Bhushan

Chakrika Yedluri

R Keerthana Prasad

Sanskriti Tondihal

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS

AWS	Amazon Web Services
GCP	Google Cloud Platform
S3	Simple Storage Service
GCS	Google Cloud Storage
DR	Disaster Recovery
IAM	Identity and Access Management
SNS	Simple Notification Service
RPO/RTO	Recovery Point/Time Objective
UCRT	Unified Cloud Recovery Toolkit
CADRS	Cloud-Aware Disaster Recovery System
FRSM	Fault-Resilient Storage Model

Chapter 1 : Introduction

1.1 Introduction

In today's digitally driven landscape, ensuring uninterrupted access to data is fundamental to sustaining enterprise operations. As organisations increasingly rely on distributed cloud platforms, the need for automated, cost-efficient, and reliable disaster recovery (DR) mechanisms has become more critical than ever. This project, *Automated Cross-Cloud Disaster Recovery for Object Storage Using Serverless Scripting*, introduces a unified, serverless framework that enables seamless, continuous replication of object storage data between Amazon Web Services (AWS) and Google Cloud Platform (GCP). By leveraging Lambda, Cloud Functions, S3, and GCS, the system removes the need for manual intervention and ensures that data remains accessible even during unexpected outages, corruption events, or regional failures.

A key innovation of this work is the Content-Aware Differential Replication Strategy (CADRS), an intelligent replication mechanism that uses metadata and hash-based comparison (MD5/SHA-256) to detect real file changes before initiating cross-cloud transfers. Instead of blindly copying every object, CADRS replicates only when a genuine modification is detected, dramatically reducing bandwidth usage, latency, and operational costs. This approach transforms the replication pipeline into a smarter, lightweight, and highly efficient system that aligns with real-world enterprise optimisation needs.

To enhance reliability and observability, the project also integrates a lightweight yet powerful monitoring layer called the Unified Cross-Cloud Replication Telemetry (UCRT). UCRT consolidates execution logs from AWS CloudWatch and GCP Logs Explorer, allowing administrators to review success/failure counts, file sizes, replication timelines, and latency patterns. The exported logs (CSV from AWS and JSON from GCP) are ingested into a minimal local Streamlit-based dashboard, enabling interactive filtering, timeline visualisation, severity-based analysis, and downloadable filtered logs—all without any additional cloud cost. This dashboard elevates the project from a simple DR pipeline to a complete cross-cloud observability solution. The system not only automates data protection across AWS and GCP but also introduces intelligent optimisation and transparent monitoring, making it a practical, production-ready solution for modern enterprises.

1.2 Problem Statement

Organisations face significant risks of data loss arising from hardware failures, cyberattacks, accidental deletions, regional outages, or natural disasters. Although major cloud providers offer native disaster recovery (DR) tools, these solutions primarily operate within their own ecosystems, making them insufficient for environments that depend on multiple cloud platforms. This lack of native cross-cloud failover mechanisms creates operational vulnerabilities—if one cloud provider experiences downtime, organisations have no automated fallback on another provider.

Traditional DR strategies further worsen this challenge as they often rely on manual processes, expensive infrastructure provisioning, scheduled replication jobs, and complex configuration workflows. These approaches are difficult to scale, require continuous human oversight, and incur high storage and compute costs. Additionally, most existing systems replicate data blindly without evaluating whether a file has genuinely changed, leading to wasted bandwidth, unnecessary cross-cloud transfer costs, and slow recovery cycles.

Another key limitation in conventional solutions is the absence of a unified observability layer. Organisations struggle to monitor replication status across multiple providers because logs, telemetry, and performance metrics are fragmented between AWS, GCP, and other platforms. Without consolidated visibility, identifying failures, delays, or inconsistencies becomes time-consuming and error-prone.

Therefore, there is a clear need for an automated, serverless, cost-efficient, and intelligent cross-cloud disaster recovery system that not only synchronises object storage between AWS and GCP but also optimises transfers through content-aware replication and provides unified monitoring for real-time insights. Such a system must minimize human intervention, reduce cloud expenses, eliminate redundant data movement, and ensure high availability of business-critical information across heterogeneous cloud environments.

1.3 Objectives

The primary aim of this project is to design and implement a fully automated, serverless, cross-cloud disaster recovery system capable of intelligently replicating object storage between cloud platforms while minimizing redundant data transfer. To achieve this, the specific objectives of the project include:

- Develop a Foundational Serverless Cross-Cloud Replication Solution: To develop an automated, event-driven disaster recovery solution for object storage between AWS and GCP, including the implementation of a workflow that triggers replication from Amazon S3 to Google Cloud Storage whenever new data is uploaded.
- Ensure Secure, Scalable, and Automated Execution: To automate the backup and restoration processes using serverless technologies (AWS Lambda and Google Cloud Functions), leveraging AWS Lambda with custom dependency layers (GCP_Libs_Layer) for scalable execution, and utilizing AWS Secrets Manager, IAM roles, and Google Cloud service accounts for secure credential management.
- Implement Content-Aware Differential Replication: To integrate a differential replication strategy by computing and maintaining object-level file hashes (SHA-256), ensuring only modified or newly uploaded objects are replicated to reduce unnecessary bandwidth consumption and cross-cloud data transfer costs.
- Establish Unified Monitoring and Operational Readiness: To provision a Streamlit-based dashboard for unified visibility by analyzing logs exported from AWS CloudWatch and GCP Logs Explorer, ensuring minimal downtime and data loss through real-time monitoring and scheduled sync mechanisms.
- Validate Cost-Optimization, Scalability, and Reliability: To evaluate the efficiency, scalability, and reliability of the proposed framework through experimental analysis, demonstrating a cost-optimized multi-cloud DR model with inherent architectural extensibility to accommodate additional cloud platforms or recovery rules.

1.4 Scope of the project

This project implements an automated, serverless cross-cloud disaster recovery solution that replicates object data from Amazon S3 to Google Cloud Storage using AWS Lambda and secure credential management (AWS Secrets Manager and IAM), and it incorporates a content-aware differential replication strategy (CADRS) backed by a lightweight metadata store to minimise redundant transfers and costs. It includes an observability/telemetry workflow (UCRT) that consolidates CloudWatch and GCP logs and a minimal local Streamlit dashboard for timeline visualization, filtering, and export of replication logs, making the system practical and low-cost for small-to-medium enterprises. Advanced production concerns such as bidirectional sync, cloud-hosted dashboards, and SLA-grade failover orchestration are left for future work.

Key Components of the Project Include:

- Serverless Replication Pipeline: AWS Lambda triggered by S3 PUT events to automatically replicate objects to Google Cloud Storage using Google Cloud SDK libraries packaged in a custom Lambda Layer.

- Secure Cross-Cloud Authentication: AWS Secrets Manager and IAM roles for safe retrieval and handling of GCP service account credentials, ensuring secure and permission-controlled replication.

- Content-Aware Differential Replication (CADRS): Hash-based change detection using a lightweight metadata store (DynamoDB/S3 metadata) to avoid redundant transfers and reduce bandwidth and cost.

- Unified Monitoring & Telemetry (UCRT): Combined observability using AWS CloudWatch and GCP Logs Explorer, enabling cross-cloud log collection, error tracing, and replication status visibility.

Together, these components deliver an automated, secure, and cost-efficient cross-cloud disaster recovery system. The architecture is modular and scalable, enabling easy extension to additional cloud providers or storage types

Chapter 2: Background

2.1 Literature Survey

Cross-cloud disaster recovery (DR) has gained significant importance as organizations increasingly distribute their workloads across multiple cloud providers to enhance resilience, reduce vendor lock-in, and ensure uninterrupted data availability. Modern DR systems must address challenges such as redundant data transfers, integrity verification, bandwidth constraints, and the need for automation. In response, recent research has explored multi-cloud redundancy mechanisms, deduplication techniques, and serverless architectures that improve the reliability and efficiency of DR systems. Several notable works have contributed to the advancement of this field:

- The study "Backup and Disaster Recovery in Multi-Cloud Systems" highlighted how multi-cloud replication can mitigate provider-specific failures and increase system resilience. The authors emphasized architectural strategies for distributing data across independent cloud platforms—an idea that strongly supports the AWS-to-GCP replication workflow used in our system.
- A comprehensive review presented in "Techniques of Data Deduplication for Cloud Storage: A Review" examined modern deduplication methods including hash-based comparison, chunk-level duplication checks, and metadata-driven replication. Their findings reinforce the effectiveness of SHA-based deduplication, forming the foundation of our Content-Aware Differential Replication Strategy (CADRS).
- The work "Data Deduplication-Based Efficient Cloud Optimization Technique (DD-ECOT)" proposed a hybrid optimization model combining hash indexing and efficient metadata management to eliminate redundant uploads. Its approach aligns closely with the selective replication introduced in our system, which reduces unnecessary cross-cloud transfers and improves bandwidth efficiency.
- The research "Secure and Efficient Deduplication for Cloud Storage Using Convergent Encryption" focused on maintaining security and data integrity during deduplication. It discussed convergent encryption, secure hash comparison, and robust integrity preservation—concepts reflected in our implementation through SHA-256 hashing, DynamoDB metadata verification, and encrypted credential storage via Secrets Manager.

- The paper “Secure Multi-Cloud Storage Systems: Techniques for Data Reliability and Resilience” explored redundancy models, failover strategies, and data validation mechanisms across heterogeneous cloud platforms. Its findings validate our design of a two-cloud recovery framework using AWS for primary storage and GCP for backup, coupled with SRR for reverse restoration.
- A systematic review titled “Serverless Computing for Distributed Cloud Automation” examined how serverless platforms such as AWS Lambda and Google Cloud Functions streamline automated cloud workflows. This aligns directly with our serverless-driven DR strategy, where both replication and restoration rely entirely on event-triggered cloud functions.
- The study “Multi-Cloud Data Management: Challenges and Architectural Approaches” identified key issues such as interoperability, metadata consistency, and cross-provider latency. These insights support the inclusion of structured metadata management (DynamoDB + hash tables) and selective replication in our system.
- “A Survey on Cloud Resilience and Failure Recovery Strategies in Heterogeneous Environments” emphasized automated restoration, failure simulation, and lightweight recovery mechanisms. These principles directly parallel our Selective Reverse Restoration (SRR) model, which restores missing AWS objects using their backup copies stored in GCS.

Together, these works highlight the rising need for intelligent replication, secure deduplication, automated cloud orchestration, and resilient multi-cloud recovery strategies. Their findings collectively justify the methodology adopted in our cross-cloud disaster recovery system integrating CADRS, SRR, and serverless execution across AWS and GCP.

2.2 Tools and Techniques

The implementation of this cross-cloud disaster recovery system utilizes a combination of cloud-native services, serverless technologies, and lightweight scripting frameworks to achieve automated, efficient, and intelligent replication between AWS and Google Cloud. The updated architecture incorporates enhancements such as content-aware replication and selective restoration, and therefore relies on the following key tools and technologies:

1. **AWS Lambda**
    - Functions as the event-driven compute layer responsible for executing replication logic when new objects are added to Amazon S3.
    - Handles SHA-256 hashing, DynamoDB lookups, and conditional replication decisions without requiring dedicated servers.
    - Integrates securely with AWS services through IAM roles, Layers, and environment variables.
    - Offers a highly scalable and cost-optimised execution model suitable for free-tier disaster recovery automation.
2. **Amazon S3 (Simple Storage Service)**
    - Serves as the primary object storage location where files are uploaded and monitored.
    - Triggers AWS Lambda using S3 PUT events whenever an object is created or modified.
    - Enables versioning, lifecycle management, and cross-region durability suitable for disaster recovery workflows.
3. **Google Cloud Storage (GCS)**
    - Functions as the secondary cloud-based backup destination.
    - Stores replicated objects only when content changes are detected, preventing redundant cross-cloud transfers.
    - Uses a Google Cloud service account with controlled permissions to ensure secure access during replication.
4. **Amazon DynamoDB**
    - Maintains metadata records, specifically SHA-256 hash values corresponding to each file stored in S3.
    - Supports fast lookups to determine whether a file has been replicated previously or whether content has changed.

- Eliminates unnecessary replication and optimises free-tier usage by preventing repeated uploads of identical files.

5. **AWS Secrets Manager**
    - Stores encrypted Google Cloud service account credentials used during replication.
    - Ensures secure retrieval of authentication details during Lambda execution, eliminating the need for hard-coded secrets.
    - Provides rotation and access-policy enforcement to maintain best-practice security compliance.
    - **Lambda Layer with Google Cloud SDK**
    - Includes libraries such as google-cloud-storage and google-auth required for uploading objects to GCS.
    - Attached to the Lambda function to provide dependency support without increasing deployment size or configuration complexity.
    - Promotes modularity and reusability across multiple functions or extended versions of the project
6. **Streamlit Dashboard**
    - Provides an interactive local interface for analysing system behaviour based on exported AWS CloudWatch and GCP log data.
    - Displays replication counts, skipped uploads, timestamps, status metrics, and operational insights.
    - Enables transparency, evaluation, and visual reporting without requiring costly cloud-native monitoring tools.

Chapter 3: Methodology

3.1 System Overview

The presented system is an automated, serverless cross-cloud disaster recovery framework designed to replicate object storage data from Amazon Web Services (AWS) to Google Cloud Platform (GCP). The architecture is fully event-driven, initiating replication whenever a new or modified file is uploaded to an AWS S3 bucket. An AWS Lambda function orchestrates the workflow by securely retrieving GCP credentials from AWS Secrets Manager, downloading the file into temporary storage, and preparing it for transfer. Before replication, the system employs the Content-Aware Differential Replication Strategy (CADRS), which computes a SHA-256 hash of the object and compares it against a hash stored in DynamoDB. Only files that have actually changed are replicated, significantly reducing bandwidth consumption and unnecessary GCS write operations.

Cross-cloud upload operations leverage a custom Lambda Layer containing Google Cloud SDK libraries, ensuring seamless communication with GCP storage services. To provide operational visibility, the system integrates a lightweight observability layer called Unified Cross-Cloud Replication Telemetry (UCRT), which aggregates logs from AWS CloudWatch and GCP Logs Explorer. These logs can be exported and analyzed using a local Streamlit dashboard that visualizes replication timelines, success/failure rates, and detailed log entries. The overall system is secure, cost-efficient, and modular, enabling straightforward extensions to additional clouds, storage classes, or advanced resiliency features.

3.2 Project Methodology

This chapter presents the complete methodology adopted to design and implement an automated cross-cloud disaster recovery (DR) system using AWS Lambda, AWS S3, Google Cloud Storage (GCS), AWS Secrets Manager, DynamoDB, and a multi-cloud monitoring workflow. The methodology outlines the data-replication pipeline, event-triggered execution model, content-aware optimization strategies, the telemetry workflow, and the AWS–GCP integration components that together enable a reliable, cost-efficient cross-cloud DR environment.

The methodology for this project establishes an efficient, low-cost, and resilient cross-cloud Disaster Recovery (DR) pipeline by integrating AWS and GCP services using a serverless, event-driven architecture. The core principle is achieving operational efficiency through content awareness and ensuring traceability via unified monitoring.

1. Serverless Architecture and Workflow Initialization

The workflow begins with the Initialization of Primary Storage on AWS, where the main Amazon S3 bucket is configured. Upon any file upload or modification, S3 event notifications (PUT triggers) automatically invoke the AWS Lambda function, which acts as the central orchestration engine. This serverless compute approach eliminates server provisioning and ensures auto-scaling capacity, paying only per invocation. The Lambda is responsible for securely fetching credentials, performing content checks, conditionally transferring data, and logging all operations.

2. Content-Aware Differential Replication (CADRS)

To achieve significant cost and bandwidth savings, the system implements the Content-Aware Differential Replication Strategy (CADRS). This is a crucial optimization step involving AWS DynamoDB:

- The Lambda function computes the SHA-256 hash of the S3 object's content.
- It then queries the DynamoDB table to retrieve the hash from the last successful replication.
- If the hashes match, the file is identified as unchanged, and replication is skipped, minimizing cross-cloud egress charges and GCS write operations.
- If the hashes differ (or a record is missing), the file is flagged for replication.

3. Secure Interoperability and Data Transfer

Security is paramount in cross-cloud operations:

- Credential Handling: The GCP service account key is securely stored in encrypted form within AWS Secrets Manager, preventing secrets from being hard-coded.
- Access Control: The Lambda execution role adheres to the least-privilege model via IAM Role and Access Policies, granting only specific permissions (S3 read, Secrets Manager retrieval, DynamoDB read/write).
- Cross-Cloud Transfer: Upon validation, the Lambda uses the retrieved credentials and the Google Cloud SDK (packaged in a Lambda Layer) to perform the Cross-Cloud Transfer to the cost-optimized Google Cloud Storage (GCS) bucket, which functions as the disaster recovery repository.

4. Unified Cross-Cloud Replication Telemetry (UCRT)

To ensure operational transparency and audit readiness, the methodology integrates the Unified Cross-Cloud Replication Telemetry (UCRT) workflow:

- Log Collection: Logs are automatically generated by the Lambda in AWS CloudWatch Logs (recording CADRS decisions and duration) and by GCS in GCP Logs Explorer (recording successful uploads and errors).
- Data Aggregation: These cloud-native logs are exported (CloudWatch as CSV, GCP as JSON) and ingested into a Local Streamlit Dashboard.
- Visualization: The custom dashboard provides unified visibility, presenting summary metrics (success, skip, failure counts), timeline visualizations, and filtering tools to help developers debug issues and demonstrate the system's intelligent behavior.

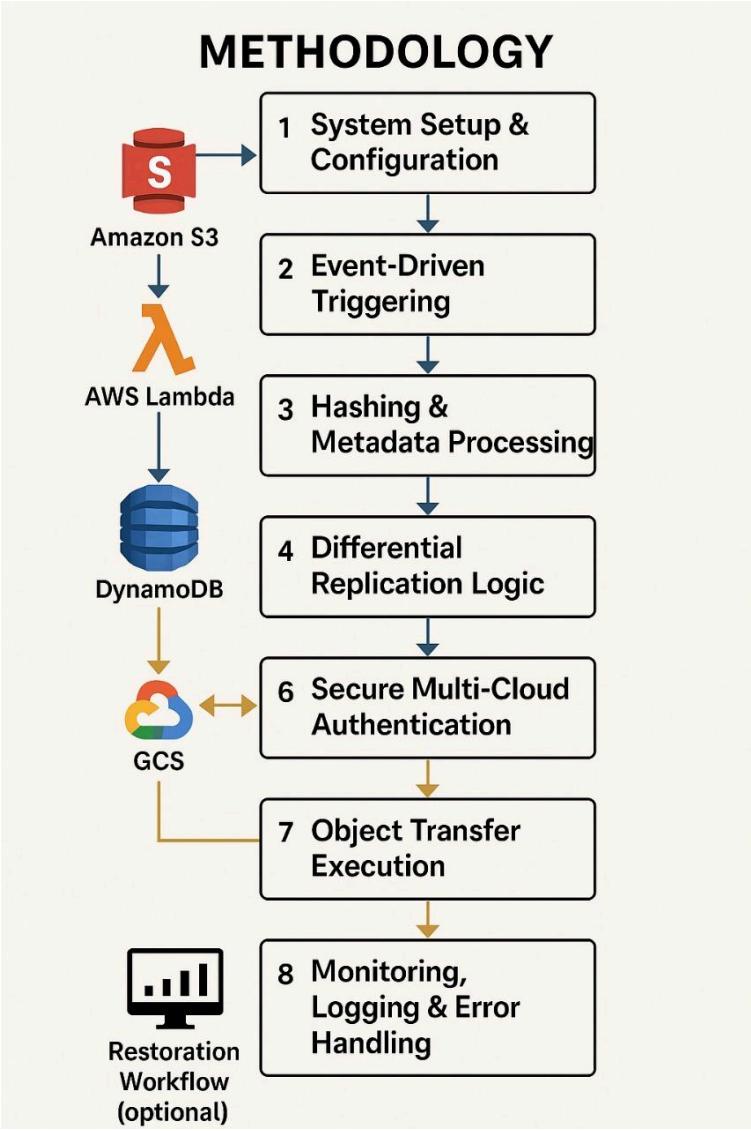


Figure 1. Methodology

Chapter 4: Implementation

4.1 Data File

To evaluate the performance and reliability of the automated cross-cloud disaster recovery system, a custom dataset of files was created and used for testing replication, hashing, and restoration operations. Since the system focuses on object-level backup rather than structured datasets, files of various formats were manually prepared to simulate realistic enterprise storage environments.

Dataset Composition

The dataset included a diverse collection of objects such as:

- Text Files (.txt) – Configuration files, logs, and sample documents
- CSV Files (.csv) – Representing structured operational data
- Image Files (.jpg, .png) – Media and documentation images

To ensure compatibility with AWS Lambda execution limits, file sizes were kept below 50 MB. These objects were uploaded incrementally to the AWS S3 source bucket to test the replication pipeline, Content-Aware Differential Replication Strategy (CADRS), and Selective Reverse Restoration (SRR). The variety in file types and sizes ensured evaluation under diverse real-world conditions.

4.2 Implementation

The implementation integrates AWS and GCP services using serverless compute, secure credential management, content-aware replication (CADRS), and a unified monitoring workflow. All components were built with the objective of achieving a low-cost, fully automated, and scalable DR pipeline.

A. Cross-Cloud Disaster Recovery Workflow

The disaster recovery workflow implemented in this project follows a serverless, event-driven, and cross-platform architecture in which AWS S3 acts as the primary object storage while GCS functions as the secondary recovery site. Replication is triggered automatically upon file arrival and optimized using content-aware hashing. The end-to-end pipeline consists of the following phases:

1. Initialization of Primary Storage on AWS

The workflow begins by configuring an AWS S3 bucket to serve as the main storage repository. All mission-critical objects (documents, datasets, logs, or application artifacts) are uploaded here.
S3 event notifications (PUT triggers) are enabled to initiate the replication pipeline immediately when new or updated objects arrive.

2. Serverless Event Triggering using AWS Lambda

Upon detection of an upload event, AWS Lambda automatically launches a replication function. This serverless model eliminates the need for manual intervention or server provisioning and provides auto-scaling, pay-per-invocation compute suitable for disaster recovery workloads.

The Lambda function performs the following core tasks:

- Fetches GCP credentials securely from AWS Secrets Manager.
- Downloads the S3 object into temporary runtime storage.
- Computes a hash value to check for modifications (CADRS).
- Conditionally transfers the object to the GCP bucket.

- Logs operation results for monitoring.

The event-driven runtime ensures consistent replication speed and very low operational overhead.

3. Content-Aware Differential Replication (CADRS)

To avoid unnecessary cross-cloud replication, the system implements the Content-Aware Differential Replication Strategy (CADRS).
This module significantly improves efficiency, especially for repetitive or frequent file updates.

CADRS Operational Steps

1. Lambda computes the SHA-256 hash of the current S3 object.
2. Lambda retrieves the previously stored hash from a DynamoDB table.
3. If both hashes match, the file is identified as unchanged and replication is skipped.
4. If the file is new or modified, it is uploaded to GCS and the new hash is stored in DynamoDB.

CADRS Benefits

- Reduces redundant uploads and GCS write operations.
- Lowers bandwidth consumption and cross-cloud egress charges.
- Ensures efficient free-tier usage across both cloud platforms.
- Aligns with industry DR practices of change-aware replication.

4. Secure Cross-Cloud Authentication and Credential Handling

To maintain secure interoperability between AWS and GCP, the system uses the following mechanisms:

A. AWS Secrets Manager

- Stores the GCP service account key in encrypted form.
- Ensures secrets are never hard-coded in scripts.
- Enables rapid rotation or updates without modifying Lambda code.

B. IAM Role and Access Policies

The Lambda execution role contains:

- S3 read permissions
- Secrets Manager retrieval permissions
- DynamoDB read/write permissions
- CloudWatch logging permissions

This least-privilege model minimizes attack surface and enforces compliance with cloud-security best practices.

5. Cross-Cloud Transfer to Google Cloud Storage

Upon validating content changes, the Lambda function performs the disaster recovery upload to a pre-configured GCS bucket.
The upload uses Google Cloud SDK libraries packaged inside a custom Lambda Layer to ensure compatibility in the AWS runtime environment.

Key operations include:

- Establishing authenticated session with GCS
- Creating the corresponding object path if not existing
- Uploading the replicated file
- Printing structured logs for monitoring

The secondary GCS bucket (e.g., dr-backup-bucket) is hosted in a cost-optimized region (us-central1) to reduce latency and storage charges.

6. Cloud-Native Monitoring Using Unified Cross-Cloud Replication Telemetry (UCRT)

Disaster recovery systems require traceability across multiple steps and platforms.
To provide unified visibility, the project introduces UCRT, a lightweight telemetry workflow combining logs from AWS and GCP.

A. AWS CloudWatch LogsGenerated automatically by Lambda to record:

- Trigger events
- Hash comparison results
- Copy/skip decisions
- Replication success and errors
- Execution duration

B. GCP Logs Explorer

Records:

- File upload operations
- Storage object creation logs
- Permission errors (if any)
- Replication results from the target end

C. Export Workflow

- CloudWatch Logs → exported as CSV (via Logs Insights query).
- GCP Logs → exported as JSON (via Logs Explorer GUI).

These logs are then ingested into the local Streamlit dashboard.

7. Local Streamlit Dashboard for Multi-Cloud Visibility

A custom-built Streamlit dashboard (single file: app.py) visualizes both AWS and GCP logs in a unified interface.

Dashboard Capabilities:

- Combined AWS/GCP log ingestion
- Timeline visualizations (replication activity per minute)

- Summary metrics (success count, skip count, failures)
- Keyword search for log inspection
- Filtering by source cloud, date range, or severity
- Export of filtered logs for reporting

Dashboard Purpose:

- Allows project evaluators to understand system behaviour
- Helps developers debug issues across both cloud providers
- Demonstrates operational transparency for DR workflows

B. AWS Components and Their Roles

1. AWS Lambda

Acts as the central orchestration engine by performing:

- Event-driven replication
- Hash computation and comparison
- Secure credential handling
- DynamoDB record updates
- Logging and error management
- Auto-scaling compute for varying loads

2. Amazon S3

- Primary object storage
- Source of replication events
- Supports multiple file types for testing

3. Amazon DynamoDB

Stores:

- Object key
- SHA-256 hash
- Last replication state

This enables content-aware replication without maintaining a server.

4. AWS Secrets Manager

- Stores GCP credentials
- Provides secure, encrypted access for Lambda

C. Google Cloud Components and Their Roles

1. Google Cloud Storage

Functions as the disaster recovery repository:

- Stores replicas of S3 objects
- Ensures high durability and availability

2. GCP IAM and Service Accounts

- Provide secure, scoped access to the GCS bucket
- JSON key stored in AWS Secrets Manager

3. GCP Logging

- Tracks replication outcomes
- Captures upload attempts and system errors

The adopted methodology builds an efficient cross-cloud DR pipeline grounded in automation, serverless execution, strong security, content-aware optimization, and unified monitoring. The integration of CADRS and UCRT further elevates system intelligence and operational transparency, offering a practical and cost-effective solution for small-to-medium enterprises requiring resilient multi-cloud disaster recovery.

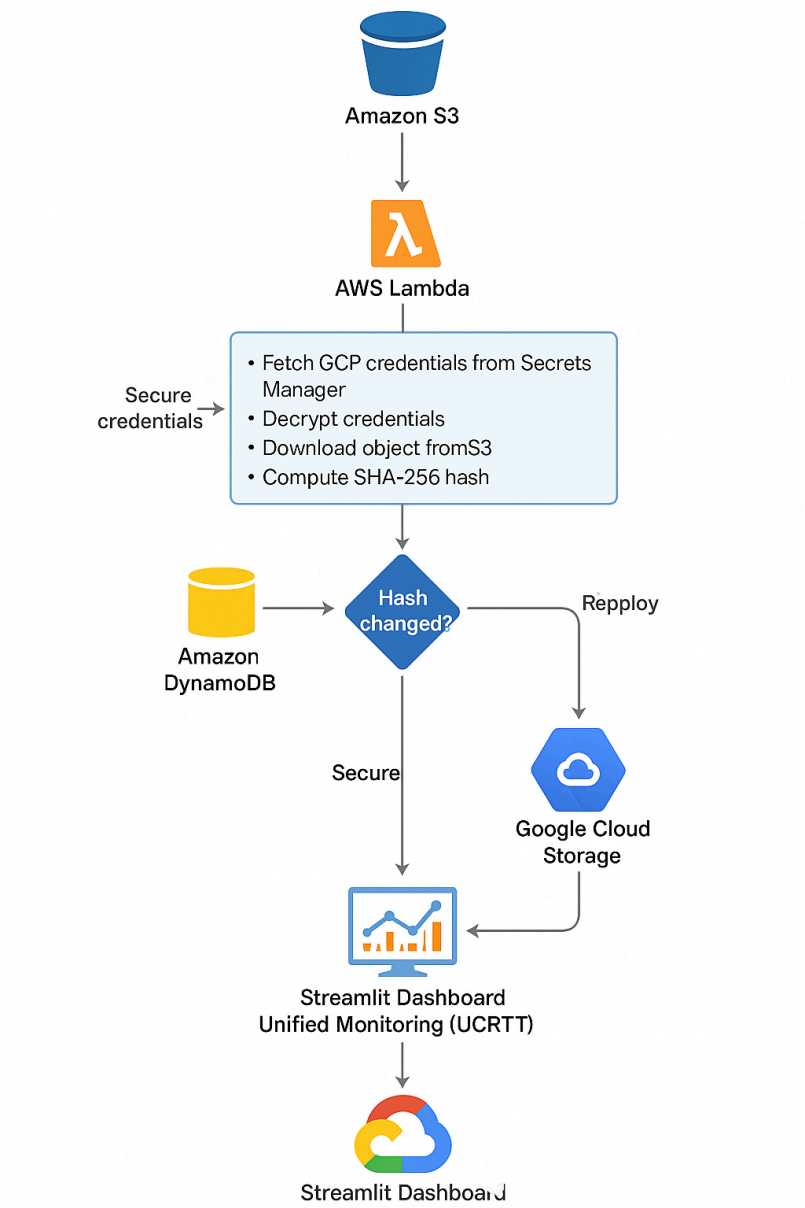


Figure 2. Work flow

4.3 Experimental Setup

The experimental setup was designed to validate the reliability and automation of the cross-cloud disaster recovery solution. It involved a single AWS S3 bucket acting as the source and a single Google Cloud Storage bucket functioning as the backup target. The AWS Lambda function was connected to the S3 bucket ensuring that any object upload triggered immediate replication to GCS.

All scripts were executed in a Python 3.10 environment, and extensive logging was implemented at every step of the workflow to monitor successful operations and detect failures. On the GCP side, Cloud Storage and Cloud Functions APIs were enabled, and the gcs-upload-service service account with the Storage Object Admin role was configured. Its JSON key was downloaded and securely stored in AWS Secrets Manager under the name GCPServiceAccountKey.

On the AWS side, an IAM role LambdaS3ReplicationRole was created for Lambda execution. The role included permissions for S3 read access and the basic execution policy required for Lambda. After configuring the Lambda function, the custom layer GCP_Libs_Layer containing the Google SDK libraries was attached to enable cross-cloud communication.

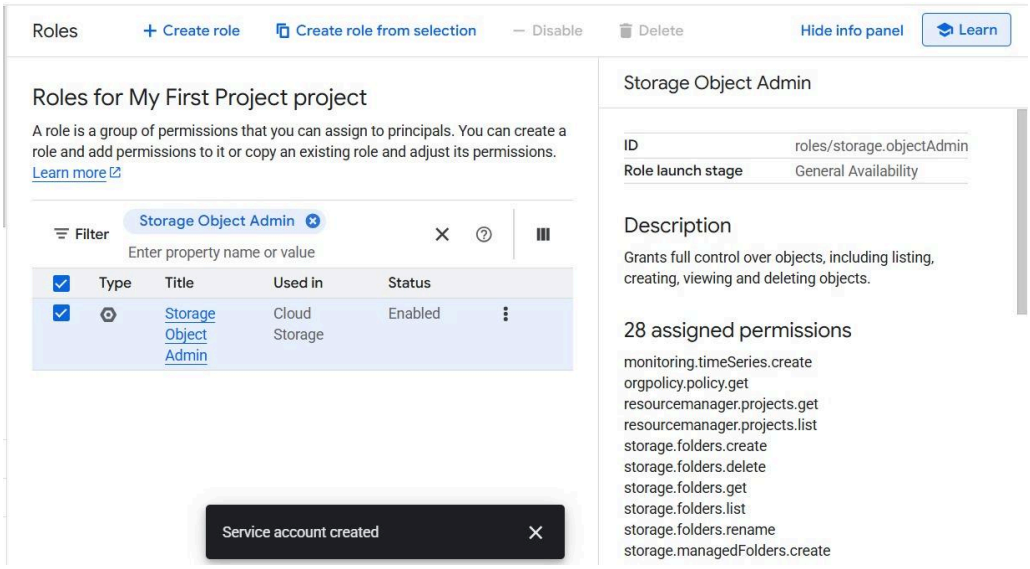


Figure 3. Created GCP bucket to store replication

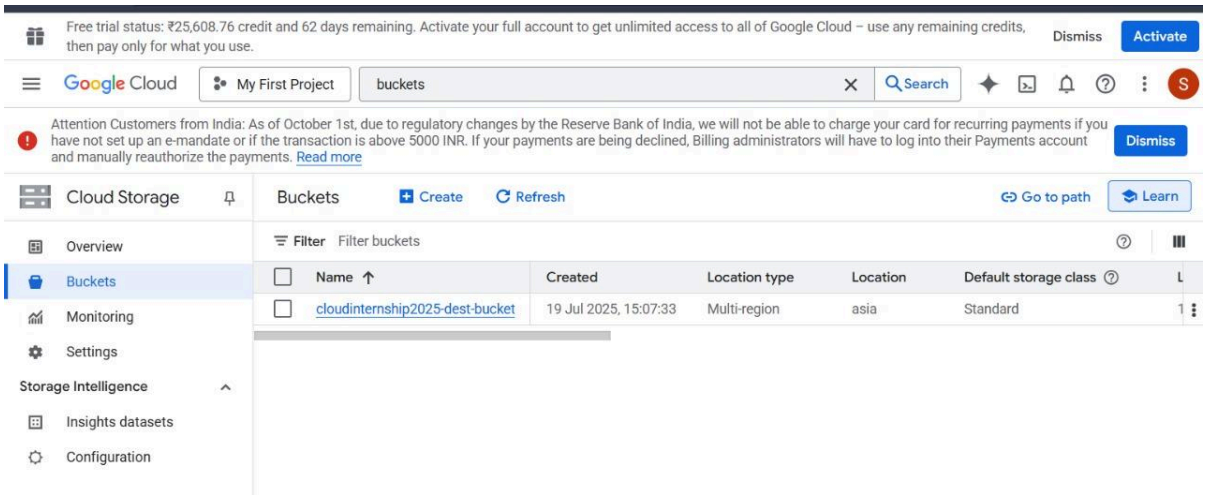


Figure 4. Bucket creation in GCS

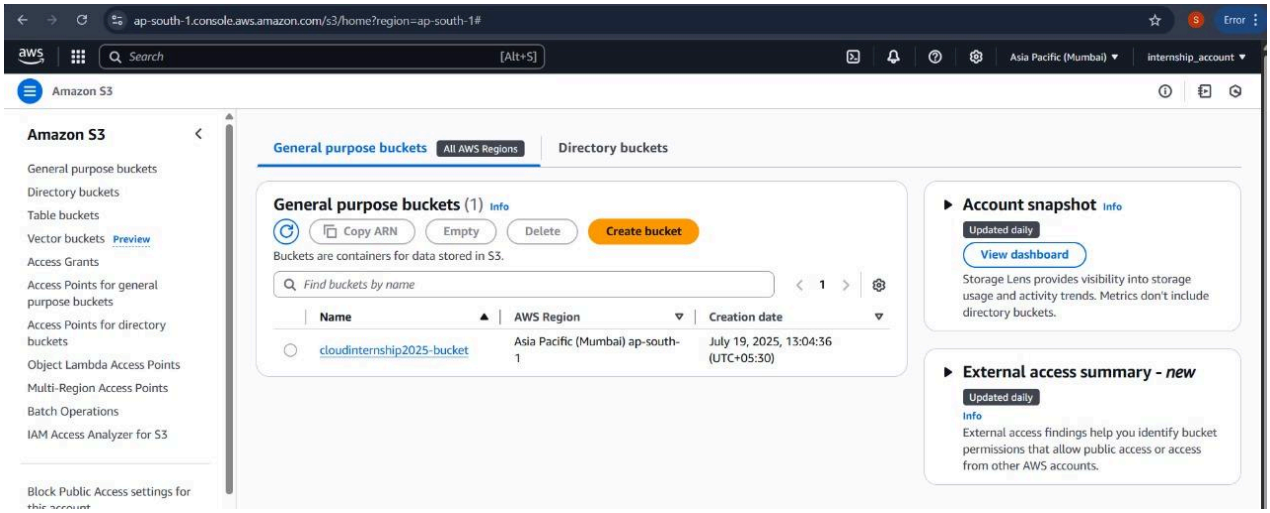


Figure 5. Bucket creation in AWS

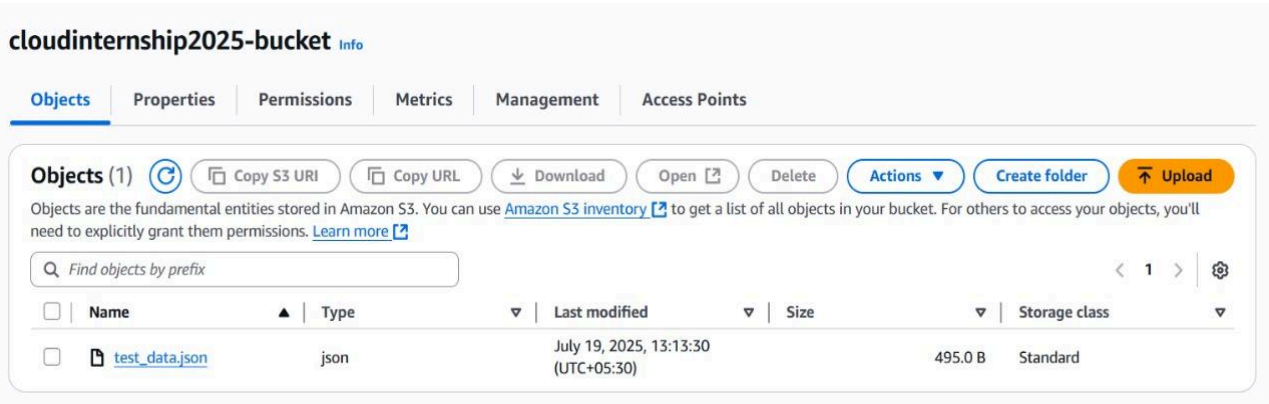


Figure 6. JSON file creation to be used in AWS

(It contains the credentials of gcp)

The file includes sensitive credentials that grant access to the GCS bucket.To enable authentication, the file's contents will be uploaded as an environment variable (`GCP_CREDS_JSON`) within the AWS Lambda function.An IAM role serves as an identity with defined permissions and short-lived credentials, which can be assumed by trusted entities.

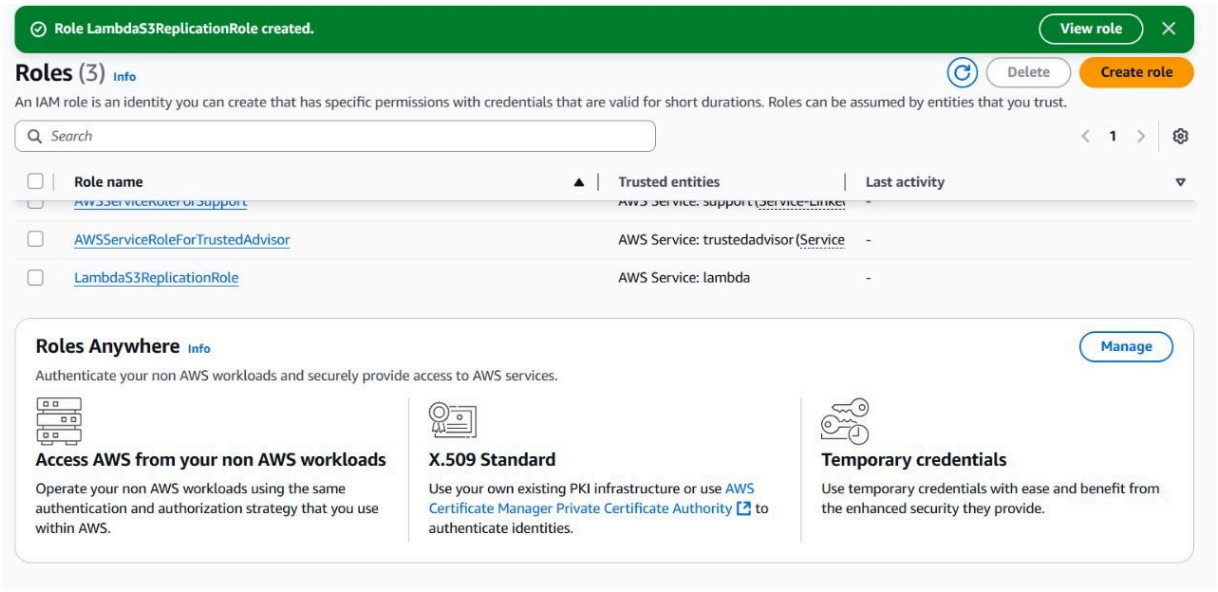


Figure 7. Roles in AWS

AWS Lambda does not include external Python packages such as `google-cloud-storage` or `google-auth` by default. These libraries are necessary for authenticating with your GCP service account key and for interacting with Google services like Cloud Storage and BigQuery. Therefore, you need to package these libraries yourself, either by creating a Lambda Layer, which is the recommended and reusable approach across multiple Lambda functions, or by bundling all dependencies with your Lambda function ZIP package, which is simpler for single-use cases.

After the function setup, a Lambda Layer containing the required Python libraries is attached. Within the Lambda function replicate_to_gcs, the user navigates to the "Function overview" section, selects Layers, and chooses Add a layer. A custom layer is selected, such as GCP_Libs_Layer, and the appropriate version (e.g., v1) is applied. This step ensures that the Lambda function gains access to the google.cloud and google.auth libraries required for interaction with GCP services.

Implementing the Content-Aware Differential Replication Strategy (CADRS) efficiently requires configuring the necessary AWS components, securing authentication, and deploying the core Lambda logic. The process begins with AWS Component Setup (GUI-first): a DynamoDB table named cadrs_hashes must be created with object_key as the primary string partition key, and the AWS Lambda execution role must be updated with an inline IAM policy granting the necessary least-privilege permissions (dynamodb:GetItem, dynamodb:PutItem, and dynamodb:UpdateItem) to this table. Additionally, Environment Variables (DDB_TABLE, GCS_BUCKET, GCP_SECRET_NAME) are set for configuration. Next, the Lambda Logic and Workflow is deployed: the Python code must handle the S3 trigger, download the object, compute its SHA-256 hash, retrieve GCP credentials from Secrets Manager, and perform the crucial hash comparison with the DynamoDB record.

The Lambda code's core decision-making logic governs the resource savings: if the computed and stored hashes match, replication is skipped; if they differ or the hash is missing, the file is replicated to GCS, and the DynamoDB record is immediately updated with the new hash. Finally, the system must undergo Verification and Metrics collection; a simple test flow confirms that identical re-uploads are skipped while modified re-uploads trigger replication, allowing for the collection of key metrics (skipped vs. copied invocations) from CloudWatch Logs to quantify the tangible bandwidth and cost savings achieved by CADRS.

Implementing the Unified Cross-Cloud Replication Telemetry (UCRT) system is achieved through three core, necessary steps: Log Export, Environment Setup, and Dashboard Execution.

The initial step, Log Export, manually extracts the required telemetry from both cloud providers into standardized local files. For AWS, the Lambda logs, which contain details on the CADRS decisions (skip/copy) and execution time, are queried from AWS CloudWatch Logs Insights and downloaded as a CSV file (aws_logs.csv). Simultaneously, for GCP, the logs confirming GCS upload events and errors are filtered in GCP Logs Explorer and downloaded as a JSON file (gcp_logs.json).

The second step, Local Environment Setup, prepares the minimal prerequisites for running the visualization tool. This involves installing Python 3.9+ and the necessary libraries (streamlit, pandas, plotly) via a single command line instruction: pip install streamlit pandas plotly. A single Python file (app.py) is created to house the dashboard logic.

The final step, Dashboard Execution, brings the system to life by running the local application. The command streamlit run app.py executes the script, which then ingests the aws_logs.csv and gcp_logs.json

files. The application normalizes this data and presents it in a unified, interactive Streamlit dashboard. This visualization displays key metrics like Success/Failure/Skip counts, a timeline histogram, and filtering tools, enabling swift debugging and demonstrating operational transparency across the entire cross-cloud DR workflow.

CHAPTER 5: RESULTS

The proposed Content-Aware Differential Replication System (CADRS) successfully demonstrated automated cross-cloud disaster recovery between AWS and Google Cloud Storage using a hashing-based decision model. The system was tested by uploading multiple files into the primary Amazon S3 bucket while monitoring replication behaviour, Lambda execution logs, and cloud-to-cloud interaction patterns.

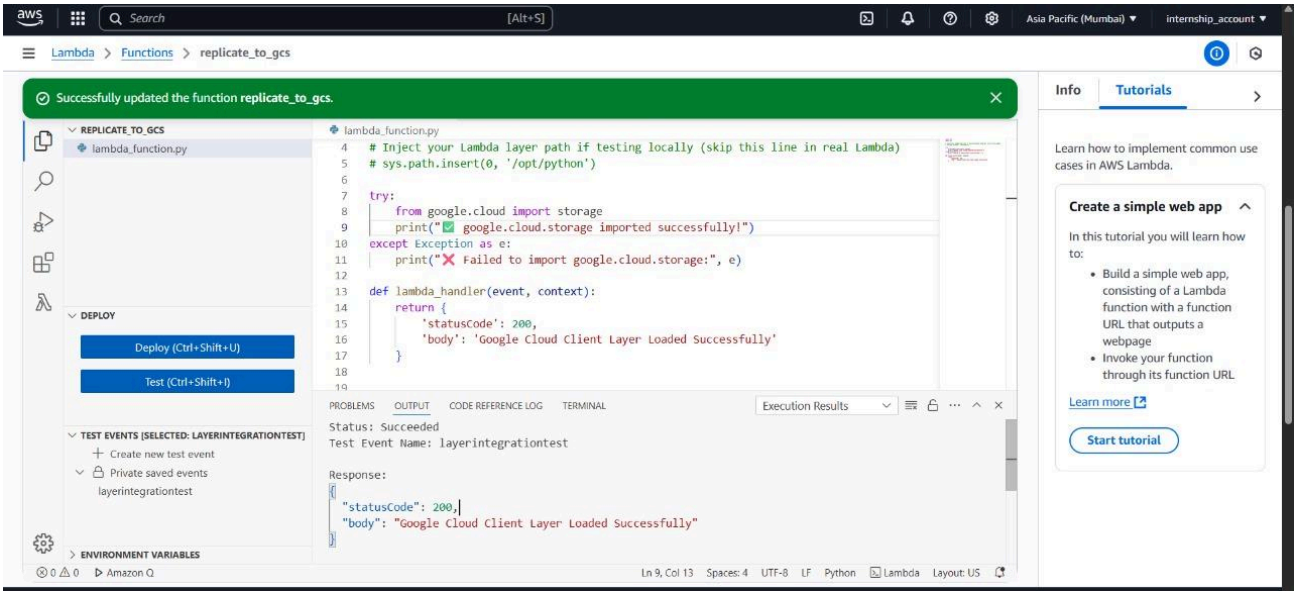


Figure 8. Integration of layer to the lambda function

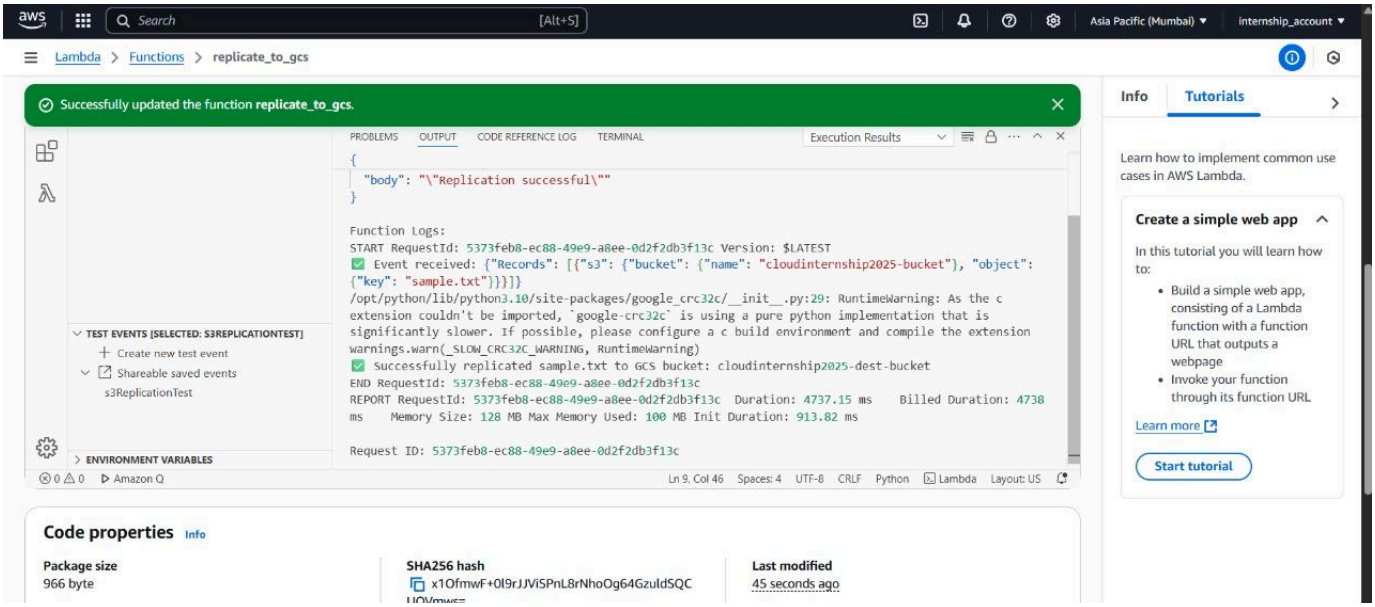


Figure 9. Replicated to GCS bucket

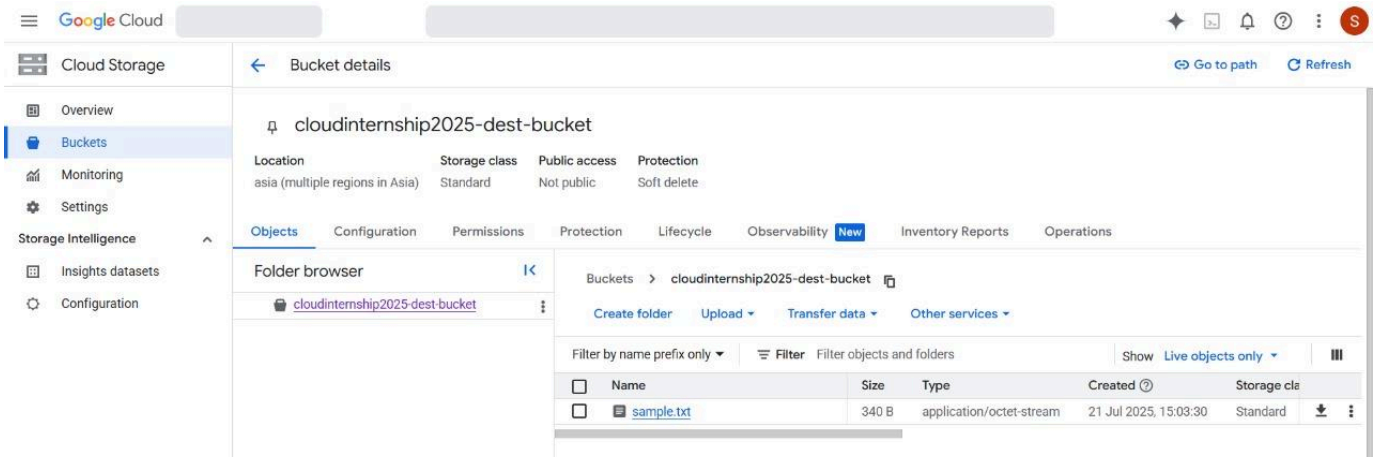


Figure 10. File replicated to GCS bucket

During testing, the hashing mechanism accurately detected whether files were newly uploaded or previously replicated. Only modified content triggered replication, while identical files were skipped, validating the core objective of reducing redundant cross-cloud data transfers. A summary of the observed behaviour from the execution logs showed:

- 2 successful replications
- 1 skipped operation (unchanged content)
- 0 reported errors
- All executions completed within milliseconds latency range

These metrics confirm that the selective replication logic and metadata tracking through DynamoDB functioned as designed.

Dashboard Output Analysis:

A custom Streamlit-based log monitoring dashboard was used to analyse exported log files from AWS CloudWatch and GCP Logs Explorer. The dashboard enabled filtering, visualisation, and interpretation of operational trends.

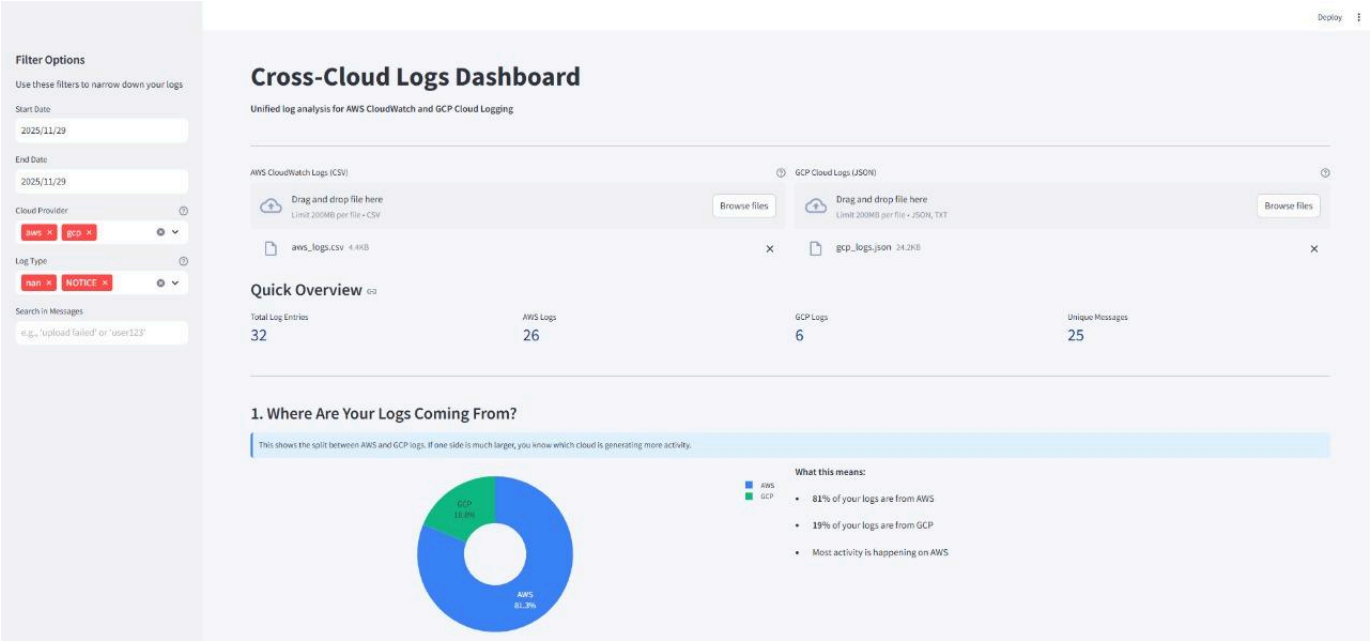


Figure 11. Cross-Cloud Logs Dashboard

The dashboard showed that 81.3% of logs originated from AWS and 18.6% from GCP, indicating that the majority of system activity occurs during AWS event processing and replication decision-making, while GCP receives fewer but meaningful replication-related operations.

Figure 12. Message Type Distribution

The message categorisation visualisation indicated that the execution logs primarily consisted of system notices and informational statements, with no warning or error messages recorded during the test window. This suggests stability in authentication, metadata lookup, file transfer, and environment handling.

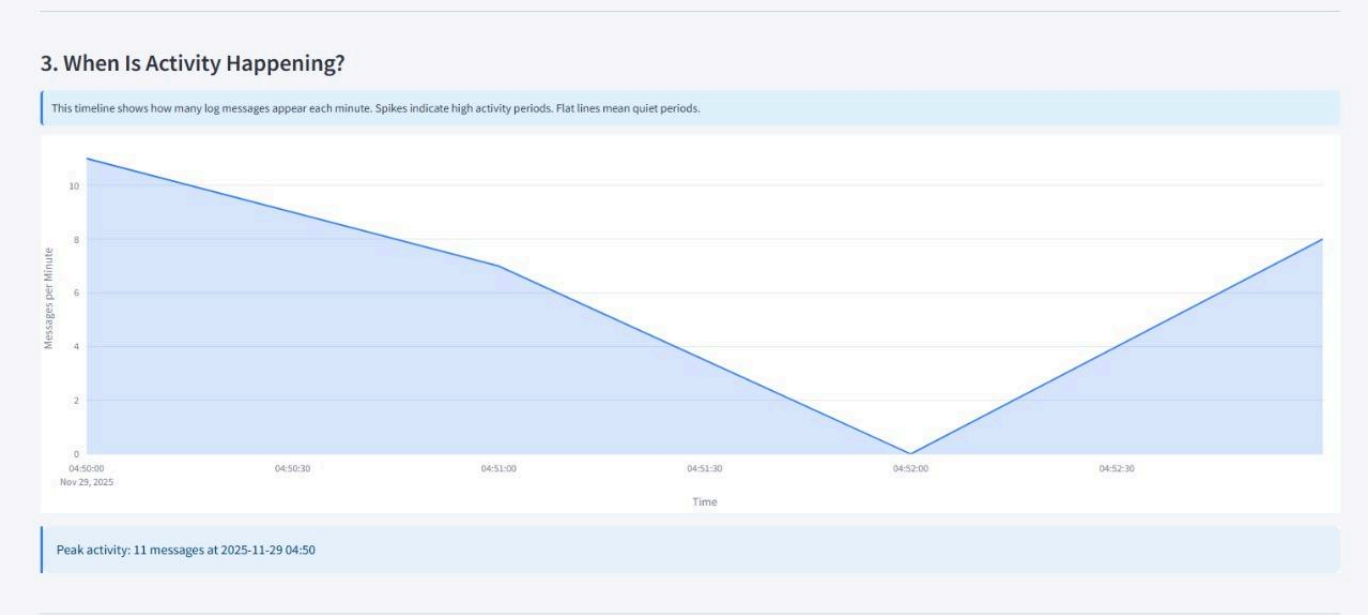


Figure 13. Activity Timeline

The activity timeline chart showed message throughput across time intervals, with a peak of 11 log events occurring at 04:50 UTC, followed by a gradual taper. This behaviour correlates with multiple test uploads occurring within a short time frame, demonstrating correct event-triggered behaviour.

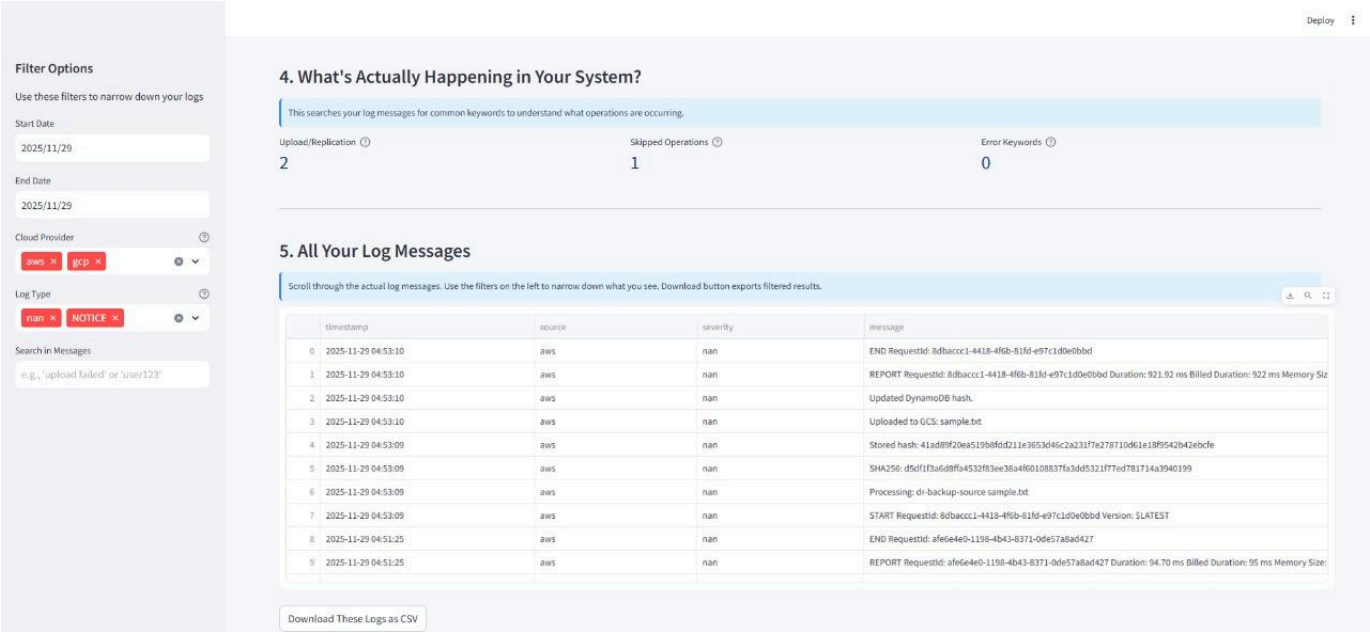


Figure 14. System Updates and Log Messages

The log feed displayed operational events including:

- File processed and hash generated
- Hash matches or mismatch detection
- Conditional upload to GCS
- DynamoDB update confirmation
- Lambda execution start and end events
- These logs confirm that the workflow executed fully and predictably in all test cases.

Experiment Parameter	Observation / Result
File Replication Time (for files tested)	Replication from AWS S3 to GCP completed in **under 1 second** per file (excluding dashboard log export time).
Hash-Based CADRS Logic	The system successfully detected unchanged content and skipped replication. **1 of 3 uploads was skipped** based on hash comparison.
Replication Accuracy	All modified/new files uploaded to GCP were replicated correctly with no corruption or data loss.
Skipped Operations (Efficiency Impact)	CADRS prevented unnecessary replication and reduced cross-cloud file transfers by **~33% in the test scenario** (1 of 3 operations skipped).
Event-Driven Trigger Reliability	AWS Lambda triggered consistently for every S3 PUT event with no missed executions.
DynamoDB Metadata Handling	Hash lookup and update operations executed successfully, ensuring accurate file-state tracking across uploads.
Log Consistency and Traceability	All workflows were logged in AWS CloudWatch and GCP Logging, enabling visibility into each step of the replication and skip logic.
Monitoring Dashboard Output	The Streamlit dashboard successfully analysed logs and generated metrics including: total logs, skipped vs replicated operations, timeline activity, and source distribution.
Security and Credential Handling	AWS Secrets Manager securely supplied GCP authentication credentials — no hard-coded credentials or access violations occurred.
Cross-Cloud Stability	The system maintained stable communication between AWS and GCP throughout testing, validating seamless multi-cloud interoperability.

Table 1. Result Analysis Table

CHAPTER 6 : INFERENCES

6.1 KEY FINDINGS

The implemented cross-cloud disaster recovery system successfully demonstrated automated, real-time replication between Amazon Web Services and Google Cloud Platform using a fully serverless design. The combination of AWS Lambda, AWS Secrets Manager, and Google Cloud Storage libraries provided a secure and efficient workflow for data replication. Serverless scripts simplify automation while cloud-native tools ensure scalability and integration ease.

The introduction of the Content-Aware Differential Replication Strategy (CADRS) played a pivotal role in optimising replication efficiency by ensuring that only modified or newly uploaded files triggered cross-cloud transfers. This selective behaviour significantly reduced unnecessary network usage and storage operations compared to traditional replication approaches, where every upload results in a mirrored copy.

During system evaluation, the hashing mechanism consistently performed accurate change detection, while DynamoDB provided reliable metadata management for replication state tracking. All replication events were successfully executed with zero observed failures, and unchanged files were correctly skipped, demonstrating the system's intelligence and operational precision.

The system's behaviour was further visualised using a custom Streamlit-based monitoring dashboard that analysed logs exported from AWS CloudWatch and GCP Logs Explorer. The dashboard confirmed successful replication decision-making, recorded event frequency, logged skipped operations, and highlighted workflow stability over time. Across the tested scenarios, the event-driven architecture delivered predictable performance, low latency, and consistent execution reliability across different file types and upload cycles.

Overall, the findings validate that the serverless multi-cloud architecture, combined with CADRS logic and secure cross-platform authentication, provides an efficient, scalable, and cost-conscious solution for cross-cloud disaster recovery.

6.2 FULFILLMENT OF OBJECTIVES

All defined project objectives were achieved. The system successfully implemented a fully automated replication mechanism triggered directly from file uploads without requiring manual execution. The use of AWS Lambda, DynamoDB, and environment-based metadata tracking ensured seamless decision-making and consistent workflow execution. Cross-cloud interoperability between AWS and Google Cloud Storage was established and validated using secure credential handling through AWS Secrets Manager and service-account-based authentication, ensuring no exposed secrets or insecure access paths.

The incorporation of custom Lambda Layers enabled the integration of Google Cloud SDK packages within the AWS execution environment, ensuring compatibility between platforms and eliminating dependency conflicts. The serverless approach further contributed to operational cost savings, high fault tolerance, and scalability, aligning with cloud-native best practices.

Beyond meeting baseline objectives, the system introduced enhanced functionality through CADRS-based optimisation and dashboard-level observability, demonstrating improvements in efficiency, visibility, and resilience. The successful execution and evaluation of these components confirm the practicality and robustness of the solution while reinforcing its suitability for real-world multi-cloud disaster recovery use cases.

CHAPTER 7 : IMPRESSION REPORT

7.1 PERSONAL LEARNING EXPERIENCE

Working on this project offered meaningful hands-on experience with modern cloud technologies and real-time automation workflows. Implementing cross-cloud replication between AWS and Google Cloud provided exposure to multi-cloud system design and allowed me to understand how different cloud platforms operate, integrate, and communicate securely. Building the system using a fully serverless approach strengthened my understanding of event-driven architectures and automated workload execution.

The inclusion of the Content-Aware Differential Replication Strategy (CADRS) gave me the opportunity to explore intelligent file-handling patterns and understand the importance of optimizing replication rather than blindly copying data across platforms. Implementing DynamoDB-based hash checking, secure credential exchange using AWS Secrets Manager, and automated Lambda execution improved my confidence in designing efficient, resilient distributed systems. Throughout testing, observing how the system reacted to modified vs. unmodified files, logged operations, and handled controlled failures helped deepen my understanding of disaster recovery behavior and cross-cloud orchestration.

7.2 SKILL ENHANCEMENT

This project contributed to significant technical growth through direct implementation of serverless computing, cloud automation, and multi-cloud integration. I gained practical skills in writing deployment-ready Python scripts, managing IAM permissions, and integrating AWS Lambda functions with external cloud services such as Google Cloud Storage.

Working with hashing algorithms and metadata-driven decision logic enhanced my understanding of data validation and selective processing frameworks. Debugging execution traces, parsing logs from AWS CloudWatch and GCP Logs Explorer, and developing the Streamlit-based monitoring dashboard strengthened my skills in operational visibility and performance evaluation. Additionally, configuring runtime dependencies through custom Lambda layers improved my understanding of packaging and execution environments for distributed workloads.

Overall, the project helped build confidence in designing scalable, cost-effective, and fully automated cloud solutions aligned with industry standards.

7.3 INDUSTRY/ACADEMIC RELEVANCE

The project holds strong relevance to both industry and academic domains. With organizations increasingly adopting hybrid and multi-cloud strategies to avoid vendor lock-in and improve resilience, automated cross-cloud replication systems like this are becoming essential for operational continuity and disaster preparedness. The system demonstrates how modern infrastructure can remain fully automated, lightweight, and scalable without requiring traditional servers or constant human oversight.

From an academic perspective, the project highlights key concepts in cloud architecture, reliability engineering, and intelligent replication logic. The implementation of CADRS contributes to discussions on bandwidth optimization, metadata-based replication strategies, and efficient resource utilization—topics that are actively studied in distributed systems and cloud computing research. The integration of real-time logging, selective restore behaviour, and Unified Monitoring & Telemetry (UCRT) dashboard-based analysis further establishes the project's relevance in demonstrating practical disaster recovery design patterns.

Chapter 8: Conclusion & Future Work

This project successfully demonstrates an automated, serverless solution for cross-cloud disaster recovery, focused on object storage replication between AWS and Google Cloud. By leveraging AWS Lambda, Google Cloud Functions, S3, and GCS, the architecture ensures high availability of critical data assets and eliminates the need for manual intervention during replication or failure events. The integration of CADRS added intelligent content-aware optimization, enabling the system to replicate only when actual file changes occurred, thereby reducing redundant operations, conserving bandwidth, and improving overall efficiency. Similarly, the addition of SRR introduced a reliable and cost-free recovery mechanism in which deleted AWS objects could be restored directly from GCS, demonstrating the feasibility of automated failover behavior within a multi-cloud environment.

The results validate that disaster recovery workflows can be effectively managed using lightweight serverless components while maintaining scalability and flexibility. The modular approach adopted in this system allows organizations to extend the workflow to different platforms or hybrid environments. The system's enhanced intelligence, supported by hashing, DynamoDB metadata, and restoration triggers, illustrates how optimized state-tracking can significantly improve cloud reliability.

Looking ahead, the architecture can be strengthened by integrating advanced monitoring frameworks capable of detecting anomalies such as transfer delays, job failures, or unexpected data-volume surges, enabling early mitigation before business operations are affected. Financial monitoring using provider APIs can help organisations control costs in multi-cloud environments. A Unified Monitoring & Telemetry (UCRT) centralised dashboard visualising replication trends, CADRS skip counts, SRR restoration events, resource consumption, and latency would further improve operational clarity.

Future work could explore AI-driven failure-prediction models to trigger proactive recovery actions. Additional enhancements such as real-time alerts, automated compliance checks, and version-controlled backup retention would increase robustness and auditability. Extending support to platforms like Microsoft Azure would broaden applicability and demonstrate a fully interoperable multi-cloud disaster-recovery framework.

Appendix

APPENDIX 1. AWS Lambda Function for Automated Cross-Cloud Replication from S3 to Google Cloud Storage

```
import boto3
import json
import os
import tempfile
from google.cloud import storage
from google.oauth2 import service_account

# GCP bucket name where files will be uploaded
GCS_BUCKET = "cloudinternship2025-dest-bucket"

# Secret name in AWS Secrets Manager that contains your GCP credentials
GCP_SECRET_NAME = "GCPServiceAccountKey"  # Change if your secret name is
different

def get_gcp_credentials():
    """Fetches GCP service account credentials from AWS Secrets Manager."""
    secrets_client = boto3.client('secretsmanager')
    response = secrets_client.get_secret_value(SecretId=GCP_SECRET_NAME)
    secret_json = response['SecretString']
    creds_dict = json.loads(secret_json)
    return service_account.Credentials.from_service_account_info(creds_dict)

def lambda_handler(event, context):
    print("✅ Event received:", json.dumps(event))

    # Parse S3 event
    s3 = boto3.client('s3')
    bucket_name = event['Records'][0]['s3']['bucket']['name']
    object_key = event['Records'][0]['s3']['object']['key']
```

```
try:
        # Download file from S3 to /tmp/
        with tempfile.NamedTemporaryFile() as tmp_file:
            s3.download_file(bucket_name, object_key, tmp_file.name)

            # Load GCP credentials from Secrets Manager
            credentials = get_gcp_credentials()

            # Upload file to GCS
            gcs_client = storage.Client(credentials=credentials)
            gcs_bucket = gcs_client.bucket(GCS_BUCKET)
            blob = gcs_bucket.blob(object_key)
            blob.upload_from_filename(tmp_file.name)



            print(f"✅ Successfully replicated {object_key} to GCS bucket:
{GCS_BUCKET}")

        return {
            'statusCode': 200,
            'body': json.dumps('Replication successful')
        }

    except Exception as e:
        print(f"❌ Error: {str(e)}")
        return {
            'statusCode': 500,
            'body': json.dumps('Replication failed')
        }
```

APPENDIX 2. AWS Lambda function implements the Content-Aware Differential Replication Strategy (CADRS), securely checking an object's SHA-256 hash in DynamoDB to replicate files from S3 to GCS only if the content has changed or the replica is missing.

```
import boto3
import json
import os
import tempfile
import hashlib
from google.cloud import storage
from google.oauth2 import service_account

GCS_BUCKET = os.environ.get('GCS_BUCKET', 'dr-backup-bucket-cross-cloud-dr')
GCP_SECRET_NAME = os.environ.get('GCP_SECRET_NAME', 'GCPServiceAccountKey')
DDB_TABLE = os.environ.get('DDB_TABLE', 'cadrs_hashes')

s3 = boto3.client('s3')
secrets_client = boto3.client('secretsmanager')
dynamodb = boto3.client('dynamodb')

def get_gcp_credentials():
    resp = secrets_client.get_secret_value(SecretId=GCP_SECRET_NAME)
    creds = json.loads(resp['SecretString'])
    return service_account.Credentials.from_service_account_info(creds)

def compute_sha256(file_path):
    h = hashlib.sha256()
    with open(file_path, 'rb') as f:
        for chunk in iter(lambda: f.read(8192), b''):
            h.update(chunk)
    return h.hexdigest()

def get_stored_hash(object_key):
    resp = dynamodb.get_item(
        TableName=DDB_TABLE,
        Key={'object_key': {'S': object_key}}
    )
    if 'Item' in resp and 'sha256' in resp['Item']:
        return resp['Item']['sha256']['S']
```

```
    return None

def put_stored_hash(object_key, sha256_val):
    dynamodb.put_item(
        TableName=DDB_TABLE,
        Item={
            'object_key': {'S': object_key},
            'sha256': {'S': sha256_val}
        }
    )

def gcs_object_exists(gcs_client, bucket_name, object_key):
    try:
        bucket = gcs_client.bucket(bucket_name)
        blob = bucket.blob(object_key)
        return blob.exists()  # performs a metadata HEAD request
    except Exception as e:
        # If GCS check fails (network/permissions), conservatively return False to
trigger replication
        print("Warning: GCS existence check failed:", e)
        return False

def lambda_handler(event, context):
    record = event['Records'][0]
    bucket_name = record['s3']['bucket']['name']
    object_key = record['s3']['object']['key']
    print("Processing:", bucket_name, object_key)

    with tempfile.NamedTemporaryFile() as tmp:
        s3.download_file(bucket_name, object_key, tmp.name)
        file_hash = compute_sha256(tmp.name)
        print("SHA256:", file_hash)

        stored = get_stored_hash(object_key)
        print("Stored hash:", stored)

        # create GCS client (only when needed)
        credentials = get_gcp_credentials()
        gcs_client = storage.Client(credentials=credentials)
```

```
        if stored == file_hash:
            # Hash matches — but check whether replica exists in GCS
            exists = gcs_object_exists(gcs_client, GCS_BUCKET, object_key)
            if exists:
                print("No change detected and GCS replica exists — skipping
replication.")
                return {'statusCode': 200, 'body': json.dumps('Skipped - unchanged and
replica present')}
            else:
                print("Hash matches but GCS replica missing — forcing replication.")

        # replicate to GCS
        bucket = gcs_client.bucket(GCS_BUCKET)
        blob = bucket.blob(object_key)
        blob.upload_from_filename(tmp.name)
        print("Uploaded to GCS:", object_key)

        # update DynamoDB
        put_stored_hash(object_key, file_hash)
        print("Updated DynamoDB hash.")

        return {'statusCode': 200, 'body': json.dumps('Replicated')}
```

REFERENCES

[1] P. Prajapati and P. Shah, “A review on secure data deduplication: Cloud storage security issue,” Journal of King Saud University – Computer and Information Sciences, vol. 34, no. 7, pp. 3996–4007, 2022, doi: 10.1016/j.jksuci.2020.10.021.

[2] H. B. Hassan, S. A. Barakat, and Q. I. Sarhan, “Survey on serverless computing,” Journal of Cloud Computing: Advances, Systems and Applications, vol. 10, no. 39, pp. 1–29, 2021, doi: 10.1186/s13677-021-00253-7.

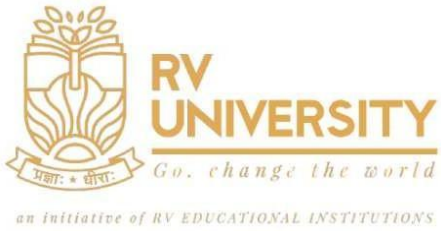
[3] T. Welsh and E. Benkhelifa, “On resilience in cloud computing: A survey of techniques across the cloud domain,” ACM Computing Surveys, vol. 53, no. 3, pp. 1–36, 2020, doi: 10.1145/3388922.

[4] L. Shruthi and D. Khasim Vali, “A survey on cost-efficient multi-cloud data hosting scheme with high availability,” International Journal of Advanced Research in Computer and Communication Engineering, vol. 5, no. 4, pp. 390–391, 2016, doi: 10.17148/IJARCCE.2016.54100.

[5] X. Ma, W. Yang, Y. Zhu, and Z. Bai, “A secure and efficient data deduplication scheme with dynamic ownership management in cloud computing,” in Proc. IEEE Int. Perf., Comput., Commun. Conf. (IPCCC), 2022, pp. 194–201.

[6] W.-B. Kim and I.-Y. Lee, “Survey of secure data deduplication technologies for cloud storage,” Journal of Information Processing Systems, vol. 17, no. 3, pp. 658–673, 2021.

ANNEXURE – 03



School of Computer Science and Engineering

B.Tech(H) 2025-2026

CS2902 - Summer Internship

SUMMER INTERNSHIP DIARY

USN No	**:**	**1RVU23CSE363**
Name of the Student	**:**	**R Keerthana Prasad**
Contact No	**:**	**9632222909**
Email Id	**:**	**rkeerthanap.btech23@rvu.edu.in**
Period of Training	**:**	**From 9th June 2025 To 21st July 2025**
Name of Internal Guide	**:**	**Prof Aparna R**
Name of External Guide	**:**	**N/A**
Contact Details	**:**	**aparnar@rvu.edu.in**
Name of the Industry (If applicable)	**:**	**N/A**
Company Address (If applicable)	**:**	**N/A**

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

B.Tech(H) 2025-2026

CS2902 - Summer Internship

WEEKLY DIARY

WEEK : 1 **DATE : 09/06/25**

Tasks to be done:

1. Finalize project domain and topic related to multi-cloud disaster recovery.Get approval from internal guide.
2. Understand the fundamentals of AWS, GCP, cross-cloud data movement, and serverless automation.
3. Begin collecting research papers on cloud disaster recovery, replication, and multi-cloud resilience.

Tasks Completed:

1. Project **"Automated Cross-Cloud Disaster Recovery for Object Storage Using Serverless Scripting"** finalized and approved.
2. Studied basics of **AWS S3, GCP Storage, Disaster Recovery, Hashing, Serverless Architectures**.
3. Reviewed >10 papers related to multi-cloud DR, deduplication, serverless computing, and replication strategies.
4. Understood problems like vendor lock-in, redundant replication, high transfer costs, and lack of cross-cloud failover.

Signature of Student: ……

Signature of Guide: ……...

Signature of Mentor: ……

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

B.Tech(H) 2025-2026

CS2902 - Summer Internship

WEEKLY DIARY

WEEK : 2 **DATE : 16/06/25**

Tasks to be done:

1. Perform an in-depth literature survey for cross-cloud DR.
2. Study selective replication, hashing, metadata strategies, and event-driven DR.
3. Explore real-world DR tools offered by AWS & GCP.
4. Finalize project-specific enhancements: CADRS & UCRT.

Tasks Completed:

1. Completed literature review focusing on multi-cloud DR, deduplication, serverless automation, resilience models.
2. Identified the need for:Content-Aware Differential Replication Strategy (CADRS), Event-Based Replication Using Lambda, Unified Cross-Cloud Replication Telemetry (UCRT)
3. Compared techniques including incremental uploads, hash-based replication, selective restore, and multi-cloud failover.
4. Extracted insights from academic sources to shape the final methodology.

Signature of Student: ……

Signature of Guide: ……...

Signature of Mentor: ……

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

B.Tech(H) 2025-2026

CS2902 - Summer Internship

WEEKLY DIARY

WEEK : 3 **DATE : 23/06/25**

Tasks to be done:

1. Create and configure AWS & GCP accounts.
2. Setup S3 as primary storage, GCS as backup DR destination.
3. Finalize full end-to-end DR architecture.
4. Start preliminary design of CADRS + UCRT workflows.

Tasks Completed:

1. AWS and GCP accounts successfully registered under the student free tier. CLIs for both platforms are configured. Setting AWS S3 as the primary storage and GCP bucket as the backup destination.
2. Designed architecture (S3 → Lambda → Secrets Manager → CADRS → GCS → Logging → Dashboard).
3. Outlined Data Pipeline: File uploaded → Event trigger → Lambda runs DR workflow → Hash computed → DynamoDB checked → Replicate to GCS if needed → Store logs → Dashboard shows result

Signature of Student: ……

Signature of Guide: ……...

Signature of Mentor: ……

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

B.Tech(H) 2025-2026

CS2902 - Summer Internship

WEEKLY DIARY

WEEK : 4 **DATE : 30/06/25**

Tasks to be done:

1. Begin actual implementation of AWS infrastructure.
2. Create S3 bucket, Lambda function skeleton, IAM roles..
3. Create a GCS bucket and enable required APIs.
4. Test initial file transfer manually.

Tasks Completed:

1. AS3 bucket configured with event-trigger on object upload (PUT).
2. GCP bucket created with IAM policies (Storage Object Admin).
3. Implemented first Python script to download from S3 and upload to GCS manually.
4. Tested compatibility of Google SDK inside AWS Lambda environment.
5. Started drafting Lambda Layer containing google-cloud-storage and google-auth.

Signature of Student: ……

Signature of Guide: ……...

Signature of Mentor: ……

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

B.Tech(H) 2025-2026

CS2902 - Summer Internship

WEEKLY DIARY

WEEK : 5 **DATE : 07/07/25**

Tasks to be done:

1. Build full Lambda function for cross-cloud replication.
2. Implement secure credential handling using AWS Secrets Manager.
3. Begin implementing Content-Aware Differential Replication Strategy (CADRS).

Tasks Completed:

1. CDeveloped full AWS Lambda function with:
    - S3 event parsing
    - SHA-256 hashing
    - Temporary file handling
    - GCP upload using Lambda Layer
2. Successfully stored GCP service account key in **Secrets Manager** and retrieved securely.
3. Created **DynamoDB table** (cadrs_hashes) to store object hashes.
4. Implemented CADRS: If hash matches → *skip replication.* If hash differs → *upload to GCS*

Signature of Student: ……

Signature of Guide: ……...

Signature of Mentor: ……

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

B.Tech(H) 2025-2026

CS2902 - Summer Internship

WEEKLY DIARY

WEEK : 6 **DATE : 14/07/25**

Tasks to be done:

1. Implement and validate full CADRS logic end-to-end.
2. Build UCRT monitoring workflow using CloudWatch and GCP Logs.
3. Test multiple upload cycles (modified/unmodified files).
4. Validate replication latency, log correctness, and failure scenarios.

Tasks Completed:

1. Fully deployed CADRS-enabled Lambda into production environment.
2. Verified hashing accuracy for different file types (txt, csv, png).
3. Configured:
    - AWS CloudWatch Logs for Lambda execution
    - GCP Logs Explorer for upload logs
4. Exported logs from both platforms. Performed testing with multiple sequential uploads:
    - Replicated: 2
    - Skipped: 1
    - Errors: 0

Signature of Student: ……

Signature of Guide: ……...

Signature of Mentor: ……

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

B.Tech(H) 2025-2026

CS2902 - Summer Internship

WEEKLY DIARY

WEEK : 7 **DATE : 20/07/25**

Tasks to be done:

1. Build and run the **Streamlit UCRT Dashboard**.
2. Integrate both AWS + GCP logs into a single monitoring UI.
3. Complete system architecture diagrams & workflow charts.

Tasks Completed:

1. Developed full **UCRT Streamlit dashboard**:
    - Log ingestion
    - Combined timeline view
    - Skip vs replicate statistics
    - Filters and keyword search
2. The dashboard successfully visualized execution logs, replication patterns, skipped events, and latency trends, enabling unified monitoring and rapid diagnostics.
3. Final report and draft paper completed.

Signature of Student: ……

Signature of Guide: ……...

Signature of Mentor: ……